Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192424193
Student Name	A. Vikranth Reddy

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])        // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Code of Conduct

I _____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____

Scalability Audit of "ShopSphere": Algorithm Analysis of Five Back-End Modules

Introduction

ShopSphere is a growing e-commerce platform. Its catalogue holds 100,000 products today and is expected to reach 1,000,000 within a year. A tenfold increase in data sounds simple, but its effect on software depends entirely on how the running time of each program grows with the amount of data. If the cost of a module is proportional to n, it becomes ten times slower. If it is proportional to n^2, it becomes a hundred times slower. If it grows like log n, it barely changes. An audit before the expansion is therefore the right time to find out which modules will cope and which will not.

As the platform's algorithm engineer, I was asked by the CTO to examine five back-end modules: the Duplicate-Listing Detector (M1), the API Rate Limiter (M2), Product Search in two versions (M3), the Recommendation Engine (M4) and the Sales Analytics Engine (M5). There is also a nightly report generator whose profiled cost is $f(n) = 3n^2 + 10n \log_2 n + 50$ operations. For each module I identify the input size and the basic operation, count that operation exactly, reduce the count to an asymptotic class, interpret the result for the platform, and finally decide which modules must be redesigned.

The same method is applied throughout. First I choose the input size n. Second I choose the basic operation, which is the step executed most often and therefore dominating the running time. Third I count that operation exactly, either by summing over loops or by solving a recurrence. Fourth I express the count in asymptotic notation (Θ, O, Ω), which removes constant factors and lower-order terms and exposes the real growth rate. Fifth I convert the count into seconds and compare it with the nightly window.

The following conventions are used in every answer:

- **Logarithms** are to base 2 unless stated, so $\log_2 100{,}000 \approx 16.61$ and $\log_2 1{,}000{,}000 \approx 19.93$.
- **Base case:** where a recurrence has no stated base case, I take $T(1) = 1$, meaning one unit of work for a problem of size 1.
- **Powers of two:** exact closed forms are derived for n a power of 2, because repeated halving then ends exactly at 1. The asymptotic classes hold for every n, and for $n = 10^6$ the formulas are accurate estimates.
- **Machine speed:** the servers execute 10^8 basic operations per second, so time in seconds equals the number of operations divided by 10^8.
- **Nightly window:** ten minutes is 600 seconds, which allows at most $600 \times 10^8 = 6 \times 10^{10}$ basic operations.

(a) Understand the Problem: Input Size and Basic Operation

The **input size** is the quantity whose growth drives the amount of work, such as the number of products in a catalogue. The **basic operation** is the single most frequently executed step. Counting it gives a measure of running time that does not depend on the programming language or on the hardware. The machine speed is only used at the end, to turn counts into seconds.

Module	**Input size n**	**Basic operation**	**Where it occurs**	**Expected growth**
M1 Duplicate detector	Number of products	Comparison of title and seller of two products	Body of the inner FOR loop	Quadratic, about $n^2/2$
M2 Rate limiter	Upper limit n on the window size	Audit(Request[j])	Body of the inner FOR loop	Linear, about n
M3-S1 Sequential search	Number of products	Comparison of the key with one product	Each step of the scan	Linear, up to n
M3-S2 Binary search	Number of entries in the sorted index	Comparison of the key with the middle entry	Once per halving step	Logarithmic, about log n
M4 Recommendation engine	Size of customer–product data	Merge step plus the recursive calls	Every recursive call	Quadratic now, n log n after redesign
M5 Sales analytics	Number of sales records	Record comparison inside the sort	Sort performed at every level	About $n \log^2 n$

M1, Duplicate-Listing Detector. The input size is the number of products n, because both loops range over the product list. The most frequently executed statement is the IF test that compares the title and the seller of Product[i] with those of Product[j]. This costs a constant number of string comparisons, and it is executed once for every pair (i, j) with $i < j$. The work therefore follows the number of pairs, and a set of n items has about $n^2/2$ pairs. I expect quadratic growth: with ten times more products there are about a hundred times more pairs.

M2, API Rate Limiter. The input size is n, because the outer loop stops when i reaches n. The only real work is the call Audit(Request[j]), so this is the basic operation. A casual reading suggests that the inner loop runs "up to n" times in each of about log n rounds, which is where the team member's n log n claim comes from. That reading is mistaken. The inner loop runs i times, and i starts at 1 and doubles every round. The early rounds are very cheap, and the total is a geometric series dominated by its last term, which is about n. I therefore expect linear growth, not n log n.

M3, Product Search. In version S1 each step of the scan compares the key with one product, and the number of steps equals the position at which the product is found, so cost grows linearly with the

catalogue. In version S2 each comparison with the middle entry of the sorted index discards half of what remains. The number of comparisons is the number of times n can be halved before one entry is left, which is about $\log_2 n$. The two versions therefore have very different growth, and the gap widens as the catalogue grows.

M4, Recommendation Engine. The input size is the size of the customer–product data. The engine is a divide-and-conquer algorithm: each call makes recursive calls on smaller pieces and then merges the partial results in linear time. The basic operation is the merge step together with the recursive calls, and the recurrence counts how many of them occur. In the current design there are four sub-problems of half size, so at the bottom of the recursion there are $4^{(\log_2 n)} = n^2$ tiny sub-problems, and I expect quadratic growth. In the redesign there are only two sub-problems of half size, giving n at the bottom and near-linear growth.

M5, Sales Analytics Engine. The input size is the number of sales records. At every level the engine sorts the records by revenue, and a sort of n records costs about n log n comparisons. The recursion has log n levels, so this adds a second logarithmic factor. I expect growth of about $n \log^2 n$, which is only slightly worse than the cost of sorting once.

(b) Analyse the Iterative Modules

Module M1: exact count of comparisons

The basic operation is the comparison "Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller", executed once per inner-loop iteration.

Step 1: the inner loop. For a fixed value of i, the inner loop runs j from i + 1 up to n − 1. The number of iterations is (n − 1) − (i + 1) + 1 = n − 1 − i. For i = 0 this is n − 1 comparisons, for i = 1 it is n − 2, and for the last value i = n − 2 it is just 1.

Step 2: the outer loop. The outer loop runs i from 0 to n − 2, so the total number of comparisons is

$C(n) = \Sigma\ (n - 1 - i)$, summed over i = 0 to n − 2.

Step 3: change of variable. Put k = n − 1 − i. When i = 0, k = n − 1, and when i = n − 2, k = 1. The sum is therefore the sum of the first n − 1 positive integers:

$C(n) = 1 + 2 + 3 + \ldots + (n - 1)$.

Step 4: closed form. The sum of the first m integers is m(m + 1)/2. With m = n − 1 we obtain

$C(n) = (n - 1)\cdot n / 2 = (n^2 - n) / 2.$

Step 5: asymptotic class. For $n \geq 2$ we have $n^2/4 \leq (n^2 - n)/2 \leq n^2/2$, so the count is bounded above and below by constant multiples of n^2. Hence $C(n) = \Theta(n^2)$.

Time and space complexity. The time complexity is $\Theta(n^2)$. It is the same in the best, average and worst case, because the loops do not depend on the data: every pair is compared whether or not duplicates exist. The space complexity is $\Theta(1)$, because besides the input the routine only stores the two indices i and j.

Meaning for ShopSphere. At today's catalogue size of 100,000 products, the routine performs 4,999,950,000 comparisons, about 5×10^9, which takes about 50 seconds. At 1,000,000 products it performs 499,999,500,000 comparisons, about 5×10^{11}, which takes about 5,000 seconds or 83 minutes. A tenfold increase in the catalogue therefore causes a hundredfold increase in time, and the nightly job will no longer fit in a ten-minute window.

Module M2: exact number of Audit operations

The basic operation is the call Audit(Request[j]) in the inner loop.

Step 1: the values taken by i. The variable i starts at 1 and is doubled after every round, so it takes the values 1, 2, 4, 8, …, 2^t, … The loop continues while i < n. If the outer loop runs k times, the last value of i used is 2^(k − 1), and this must be less than n, while the next value 2^k must be at least n. Therefore 2^(k − 1) < n ≤ 2^k, which means

$k = \lceil \log_2 n \rceil$.

Step 2: work in each round. In the round where $i = 2^t$, the inner loop FOR j = 0 TO i − 1 executes exactly $i = 2^t$ Audit calls. It does not execute n calls. This is the key observation.

Step 3: total. Summing over the rounds t = 0 to k − 1 gives a geometric series with ratio 2:

$A(n) = 2^0 + 2^1 + 2^2 + \ldots + 2^{(k-1)} = (2^k - 1)/(2 - 1) = 2^k - 1$.

Step 4: bound in terms of n. From $n \le 2^k < 2n$ we get $n - 1 \le 2^k - 1 < 2n - 1$. Therefore the count lies between $n - 1$ and $2n$, which proves

$A(n) = 2^{\lceil \log_2 n \rceil} - 1 = \Theta(n)$.

When n is itself a power of 2, then $2^k = n$ and $A(n) = n - 1$ exactly.

Is the claim of $\Theta(n \log n)$ correct? No. The team member reasoned that the outer loop runs about log n times and the inner loop runs up to n times, so the cost is at most n log n. The reasoning treats every round as if it cost n, but the inner loop runs i times, and i is small in the early rounds: 1, 2, 4, 8, and so on. Only the last round costs about n/2 to n, and all earlier rounds added together cost about the same amount again, because the sum of a geometric series is dominated by its largest term. The statement $A(n) = O(n \log n)$ is technically true, since n log n is an upper bound, but it is not tight. Because A(n) is also O(n) and $A(n) \ge n - 1$, the tight bound is $\Theta(n)$. A Θ bound must hold from both sides, and n log n fails the upper side because $A(n)/(n \log n)$ tends to zero.

Verification for n = 1,024. Here $n = 2^{10}$, so k = 10 rounds, in which i takes the values 1, 2, 4, 8, 16, 32, 64, 128, 256 and 512. When i becomes 1,024 the test $i < n$ fails and the loop stops. The numbers of Audit calls are 1, 2, 4, 8, 16, 32, 64, 128, 256 and 512, and their sum is 1,023. The formula gives $2^{10} - 1 = 1{,}023$, and this equals $n - 1$, so the derivation is confirmed. The incorrect claim would predict about $n \log_2 n = 1{,}024 \times 10 = 10{,}240$ operations, ten times more than the real count, and the gap grows as n grows.

Time and space complexity. The time complexity is $\Theta(n)$, and the space complexity is $\Theta(1)$.

Meaning for ShopSphere. At $n = 1{,}000{,}000$, $k = 20$ and $A(n) = 2^{20} - 1 = 1{,}048{,}575$ Audit calls, which takes about 0.0105 seconds. The rate limiter scales comfortably, and the only change needed is to correct the documentation: the module is linear, not n log n.

(c) Solve a Recurrence by Substitution: Search Version S2

Binary search on the sorted product index has the recurrence

$T(n) = T(n/2) + 1$, with $T(1) = 1$,

where T(n) is the number of key comparisons. The "+ 1" is the one comparison with the middle entry, and T(n/2) is the cost of searching the half that remains. To make repeated halving exact, let $n = 2^k$, so that $k = \log_2 n$.

Backward substitution. I expand the recurrence repeatedly, each time replacing the T term on the right by its own definition:

$T(n) = T(n/2) + 1$

$T(n) = [T(n/4) + 1] + 1 = T(n/2^2) + 2$

$T(n) = [T(n/8) + 1] + 2 = T(n/2^3) + 3$

$T(n) = [T(n/16) + 1] + 3 = T(n/2^4) + 4$

General pattern. After i substitutions, $T(n) = T(n/2^i) + i$. This can be confirmed by induction: replacing $T(n/2^i)$ by T(n/2^(i+1)) + 1 gives T(n/2^(i+1)) + (i + 1), which is the same form with i + 1.

Stopping condition. The recursion reaches the base case when $n/2^i = 1$, that is, when $i = k = \log_2 n$. Substituting this value of i and using $T(1) = 1$:

$T(n) = T(1) + \log_2 n =$ **$\log_2 n + 1$.**

For a general n that is not a power of 2, the halving uses the floor function and the exact value is $\lfloor \log_2 n \rfloor + 1$.

Check. $T(1) = 1$, $T(2) = T(1) + 1 = 2$, $T(4) = 3$ and $T(8) = 4$. The formula gives $1 + \log_2 8 = 4$, which agrees.

Time complexity. The closed form shows that $T(n) = \Theta(\log n)$. For n = 1,000,000, $T = \lfloor 19.93 \rfloor + 1 =$ 20 comparisons.

Space complexity of the two implementations.

Implementation	**Time complexity**	**Space complexity**	**Reason**
Iterative (loop with low, high, mid)	$\Theta(\log n)$ worst case, $\Theta(1)$ best case	$\Theta(1)$	Only three variables are stored, and no memory grows with n
Recursive (function calls itself on one half)	$\Theta(\log n)$ worst case, $\Theta(1)$ best case	$\Theta(\log n)$	Each call adds a stack frame, and the depth is $\lfloor \log_2 n \rfloor + 1$ frames

Both implementations perform the same comparisons, so their time is the same. They differ in memory. The iterative version keeps just the boundaries of the current half, so it needs constant extra space. The

recursive version keeps one stack frame for every halving until it returns, which is $\Theta(\log n)$ frames. Some compilers remove this overhead by tail-call optimisation, but it cannot be relied on.

Meaning for ShopSphere. Searching 1,000,000 products needs only about 20 comparisons, and multiplying the catalogue by ten adds only about $\log_2 10 \approx 3.3$ comparisons. The iterative version is the better choice for production because it has constant memory and avoids stack overhead.

(d) Apply the Master Theorem

The Master Theorem solves recurrences of the form T(n) = aT(n/b) + f(n), with a ≥ 1 and b > 1. The quantity n^(log_b a) measures the total cost of the leaves of the recursion tree, and f(n) is the cost of the work done outside the recursive calls at the top. The theorem compares the two:

- **Case 1:** if f(n) = O(n^(log_b a − ε)) for some ε > 0, the leaves dominate and T(n) = Θ(n^(log_b a)).
- **Case 2:** if f(n) = Θ(n^(log_b a)), every level costs about the same and T(n) = Θ(n^(log_b a) · log n).
- **Case 3:** if f(n) = Ω(n^(log_b a + ε)) for some ε > 0, and the regularity condition a·f(n/b) ≤ δ·f(n) holds for some δ < 1, the top dominates and T(n) = Θ(f(n)).

Module M4: current design and proposed redesign

Quantity	**Current design: T(n) = 4T(n/2) + n**	**Redesign: T(n) = 2T(n/2) + n**
a	4	2
b	2	2
f(n)	n	n
log_b a	$\log_2 4 = 2$	$\log_2 2 = 1$
n^(log_b a)	n^2	n
Comparison	n = O(n^(2 − ε)) with ε = 1	n = Θ(n)
Case	Case 1	Case 2
Result	$\Theta(n^2)$	Θ(n log n)

Current design. Here a = 4, b = 2 and f(n) = n. Then log_b a = $\log_2 4 = 2$ and n^(log_b a) = n^2. Since f(n) = n grows polynomially more slowly than n^2 (it is O(n^(2 − ε)) with ε = 1), Case 1 applies and T(n) = $\Theta(n^2)$.

Redesign. Here a = 2, b = 2 and f(n) = n. Then log_b a = $\log_2 2 = 1$ and n^(log_b a) = n. Since f(n) = Θ(n), which is exactly of the same order, Case 2 applies and T(n) = Θ(n log n).

Cross-check with recursion trees (T(1) = 1, L = $\log_2 n$ levels). For the current design, level i has 4^i sub-problems of size $n/2^i$, each costing $n/2^i$ to merge, so the level cost is $4^i \cdot n/2^i = n \cdot 2^i$. Summing over levels i = 0 to L − 1 gives n(2^L − 1) = n(n − 1). The bottom level has 4^L = n^2 leaves of cost 1 each. The total is $n^2 + n(n-1) = 2n^2 - n$, which is $\Theta(n^2)$. As a check, T(2) = 4·1 + 2 = 6 and 2·4 − 2 = 6. For the redesign, level i has 2^i sub-problems, each costing $n/2^i$, so every level costs exactly n. With L levels and n leaves, the total is n $\log_2 n$ + n, which is Θ(n log n). As a check, T(2) = 2·1 + 2 = 4 and 2·1 + 2 = 4.

What happens when the data doubles. For the current design, $T(2n) = 2(2n)^2 - 2n = 8n^2 - 2n$, which is about four times $T(n) = 2n^2 - n$. The running time quadruples every time the data doubles. For the redesign, $T(2n)/T(n) = 2(\log_2 n + 2)/(\log_2 n + 1)$. At $n = 10^6$ this is about 2.1, so doubling the data costs only slightly more than double, which is close to linear scaling.

Meaning for ShopSphere. The replicated partitions cost the engine a whole polynomial factor. At $n = 10^6$ the current design needs about 2×10^{12} operations and the redesign about 2.1×10^7, a difference of roughly 100,000 times. The redesign is clearly the right choice.

Module M5: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2 and f(n) = n log n, and n^(log_b a) = n^(log₂ 2) = n. Each case is checked in turn.

- **Case 1** requires f(n) = O(n^(1 − ε)), that is, f smaller than n. This fails, because n log n is larger than n.
- **Case 2** requires f(n) = Θ(n). This fails, because n log n grows faster than any constant multiple of n.
- **Case 3** requires f(n) = Ω(n^(1 + ε)) for some ε > 0. This also fails, because log n grows more slowly than every positive power n^ε, so n log n is smaller than n^(1 + ε) for large n.

The function f(n) = n log n lies in the gap between n and n^(1 + ε), so **none of the three standard cases applies.**

Extended Master Theorem. If $f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ for some $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$. Here $f(n) = n \cdot \log^1 n$, so k = 1, and

$T(n) = \Theta(n \log^2 n)$.

Verification by recursion tree. Let $L = \log_2 n$. Level i has 2^i nodes, each of size $n/2^i$ costing $(n/2^i) \cdot \log(n/2^i) = (n/2^i)(L - i)$. The level total is therefore $n(L - i)$. Summing over the levels i = 0 to L − 1:

$\Sigma\, n(L - i) = n(L + (L - 1) + \ldots + 1) = n \cdot L(L + 1)/2.$

The n leaves add n, so $T(n) = n \cdot L(L + 1)/2 + n = (n \log^2 n + n \log n)/2 + n$, which is $\Theta(n \log^2 n)$. This agrees with the extended theorem.

Meaning for ShopSphere. For $n = 10^6$ ($L \approx 19.93$), $T \approx 10^6 \times (397.3 + 19.9)/2 + 10^6 \approx 2.10 \times 10^8$ operations, about 2.1 seconds. The engine is only one logarithmic factor slower than sorting once, and this is perfectly acceptable at this scale.

(e) Analyse Best, Average and Worst Cases

Search version S1: sequential search

Case	Input condition	Comparisons
Best	The searched product is the first in the catalogue	1
Average (product exists)	The product is equally likely to be at any position	(n + 1)/2
Worst	The product is the last one, or is not in the catalogue	n

Average-case derivation with probability p. Let p be the probability that the searched product exists in the catalogue.

If the product exists, it is equally likely to be at any of the n positions, so it is at position i with probability p/n, and finding it takes i comparisons. The expected cost of the successful case is

$\Sigma\, i \cdot (p/n)$, summed over i = 1 to n, which equals $(p/n) \cdot n(n + 1)/2 = p(n + 1)/2$.

If the product does not exist, which has probability $1 - p$, the scan examines the whole catalogue, costing n comparisons. The expected cost of this case is $n(1 - p)$.

Adding the two contributions:

$C_{avg} = p(n + 1)/2 + n(1 - p) = [p(n + 1) + 2n(1 - p)]/2 =$ **$[n(2 - p) + p]/2$.**

As a check, p = 1 gives (n + 1)/2, the average for a search that always succeeds, and p = 0 gives n, the cost of a search that always fails, so the formula agrees with the table.

Numerical evaluation for n = 1,000,000 and p = 0.5.

$C_{avg} = 0.5 \times 1{,}000{,}001/2 + 1{,}000{,}000 \times 0.5 = 250{,}000.25 + 500{,}000 =$ **750,000.25 comparisons.**

At 10^8 operations per second this is 0.0075 seconds per search. The average case is only a constant factor of about 3/4 better than the worst case, so S1 is still $\Theta(n)$. A single search is fast enough, but this cost is multiplied by every search a busy shopping platform handles each second.

Search version S2: binary search

Case	Input condition	Comparisons
Best	The key equals the first middle entry examined	1
Average	The key is at a random position (or absent)	About $\log_2 n - 1$ if found, about $\log_2 n$ if absent
Worst	The key is found only when one entry remains, or is not in the index	$\lfloor \log_2 n \rfloor + 1$

The best case occurs when the very first middle entry is the searched product, so the search ends after one comparison. The worst case occurs when the key is found only at the last level, or when it is absent:

the index must then be halved until nothing is left, which takes $\lfloor \log_2 n \rfloor + 1$ comparisons, in agreement with the closed form of part (c).

For the average case, consider a successful search in an index of $n = 2^k - 1$ entries, which forms a perfectly balanced decision tree. There are $2^{(l-1)}$ entries at level l (for $l = 1$ to k), and finding an entry at level l takes l comparisons. The average is

$C_{avg} = (1/n) \Sigma\, l \cdot 2^{(l-1)} = ((k-1)2^k + 1)/(2^k - 1) \approx k - 1 \approx \log_2 n - 1.$

Most entries are in the last levels of the tree, so a typical search costs almost the maximum.

For n = 1,000,000, the best case is 1 comparison, the average is about 19 and the worst case is 20. All of these are tiny compared with S1, which needs up to 1,000,000 comparisons in the worst case and 750,000 on average at $p = 0.5$. Binary search is more than 37,000 times cheaper on average. ShopSphere should therefore use S2, or a hash or balanced-tree index, for every product search. Its only requirement is that the index is kept sorted, and the cost of keeping it sorted is paid once per update, not once per search.

(f) Prove an Asymptotic Bound

Definition. A function f(n) is $\Theta(g(n))$ if there exist positive constants c_1 and c_2 and a threshold n_0 such that $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$. We must prove this for $f(n) = 3n^2 + 10n \log_2 n + 50$ and $g(n) = n^2$.

Step 1: the upper bound, $f(n) = O(n^2)$. For every $n \ge 1$, three simple facts hold. First, $\log_2 n \le n$, so $10n \log_2 n \le 10n \cdot n = 10n^2$. Second, $n^2 \ge 1$, so $50 \le 50n^2$. Third, the first term is already $3n^2$. Adding the three inequalities:

$f(n) = 3n^2 + 10n \log_2 n + 50 \le 3n^2 + 10n^2 + 50n^2 = 63n^2$, for all $n \ge 1$.

So the upper bound holds with $\mathbf{c_2 = 63}$ and $n_0 = 1$.

Step 2: the lower bound, $f(n) = \Omega(n^2)$. For every $n \ge 1$, the terms $10n \log_2 n$ and 50 are non-negative, so removing them cannot increase the value:

$f(n) = 3n^2 + 10n \log_2 n + 50 \ge 3n^2$, for all $n \ge 1$.

So the lower bound holds with $\mathbf{c_1 = 3}$ and $n_0 = 1$.

Step 3: conclusion. Combining both steps,

$3n^2 \le f(n) \le 63n^2$, for all $n \ge n_0 = 1$,

and therefore $\mathbf{f(n) = \Theta(n^2)}$ with $c_1 = 3$, $c_2 = 63$ and $n_0 = 1$.

The constants do not have to be optimal; any valid triple proves the claim. A tighter upper constant also works: for $n \ge 64$ we have $10n \log_2 n + 50 \le n^2$, so $f(n) \le 4n^2$ with $n_0 = 64$. For example, at $n = 64$ the left side is $10 \cdot 64 \cdot 6 + 50 = 3{,}890$, which is below $64^2 = 4{,}096$.

Step 4: f(n) is not O(n). I use proof by contradiction. Suppose $f(n) = O(n)$. Then there would exist a constant $c > 0$ and a threshold n_0 such that $f(n) \le c \cdot n$ for all $n \ge n_0$. From the lower bound of Step 2, $f(n) \ge 3n^2$ for all $n \ge 1$, so for every $n \ge n_0$,

$3n^2 \le f(n) \le c \cdot n$.

Dividing by n gives $3n \le c$, that is, $n \le c/3$ for every $n \ge n_0$. This claims that every integer $n \ge n_0$ is at most $c/3$, which is false. For instance, take $n = \max(n_0, \lceil c/3 \rceil + 1)$; then $n > c/3$, which contradicts the inequality. Hence no such c and n_0 exist and f(n) is not O(n). Equivalently, $f(n)/n$ tends to infinity.

Meaning for ShopSphere. The report generator is truly quadratic. The n log n term and the constant 50 become negligible as n grows, and the cost is governed by $3n^2$. At $n = 10^6$ the cost is about 3×10^{12} operations, which is about 30,000 seconds or 8.3 hours. A cost like this can only be fixed by changing the algorithm, because faster hardware or tuning of constants cannot compensate for a hundredfold increase.

(g) Scalability Decision

The table uses the formulas derived above, with $n = 10^6$ and $\log_2 n \approx 19.93$. Time equals operations divided by 10^8, and the nightly window allows 6×10^{10} operations.

Module	**Complexity**	**Operations at n = 10^6**	**Time**	**Within 10 minutes?**
M1 Duplicate detector	$\Theta(n^2)$, $n(n-1)/2$	about 5.0×10^{11}	about 5,000 s (83 min)	No (8.3 times over)
M2 Rate limiter	$\Theta(n)$, $2^{\lceil \log n \rceil} - 1$	1,048,575	about 0.0105 s	Yes
S2 Binary search	$\Theta(\log n)$, $\lfloor \log n \rfloor + 1$	20	about 2×10^{-7} s	Yes
M4 current design	$\Theta(n^2)$, $2n^2 - n$	about 2.0×10^{12}	about 20,000 s (5.6 h)	No (33 times over)
M4 redesign	$\Theta(n \log n)$, $n \log n + n$	about 2.09×10^7	about 0.21 s	Yes
M5 Sales analytics	$\Theta(n \log^2 n)$	about 2.10×10^8	about 2.1 s	Yes
Report generator f(n)	$\Theta(n^2)$, $3n^2 + \ldots$	about 3.0×10^{12}	about 30,000 s (8.3 h)	No (50 times over)

For comparison, at today's size of $n = 10^5$ the same quadratic modules take about 50 seconds (M1), about 200 seconds (M4 current) and about 300 seconds (report generator). All three fit in the window today, but each becomes 100 times slower after the expansion. The efficient modules change little: M2 rises from about 0.0013 seconds to 0.0105 seconds, M5 from about 0.15 seconds to 2.1 seconds, and the M4 redesign from about 0.02 seconds to 0.21 seconds.

Modules that fail the 10-minute window. Three components fail: M1 (about 83 minutes), the current M4 design (about 5.6 hours) and the nightly report generator (about 8.3 hours, which is not one of the five modules, but its $\Theta(n^2)$ cost makes it fail as well). All three are quadratic, and the lesson of the audit is that this growth class cannot survive a tenfold expansion.

Recommendations.

M1 must be redesigned. The all-pairs comparison should be replaced by hashing. Each product is looked up in a hash table keyed by the pair (title, seller). If the key is already present, the product is marked as a duplicate, otherwise the key is inserted. The expected cost of each lookup is constant, so the whole job takes expected $\Theta(n)$ time, about 10^6 operations or 0.01 seconds at $n = 10^6$, in exchange for $\Theta(n)$ extra memory. An alternative that needs little extra memory is to sort the products by (seller, title) in $\Theta(n \log n)$ and compare only neighbours. The best long-term fix is a unique database index on (title, seller), which rejects duplicates at upload time and makes the nightly job almost unnecessary.

M4 should adopt the proposed redesign. Removing the replication changes the recurrence from $4T(n/2) + n$ to $2T(n/2) + n$ and the complexity from $\Theta(n^2)$ to $\Theta(n \log n)$. Time at $n = 10^6$ falls from about 5.6 hours to about 0.2 seconds. If replicated partitions are still needed for reliability, they should be kept at the storage level and not inside the computation.

The report generator should be redesigned. Its quadratic term dominates, so the profiler should be used to locate the nested loop or pairwise step responsible, which can usually be replaced by sorting or by hash-based grouping, reaching $\Theta(n \log n)$ or $\Theta(n)$.

M2, S2 and M5 can stay. M2 is linear with constant space, and only its documentation should be corrected from n log n to $\Theta(n)$. Search should use S2, or a hash index, and S1 should be retired, since it needs up to a million comparisons per search against about 20. M5 uses about two seconds, far inside the window. If further improvement is ever required, sorting once and reusing the order, instead of re-sorting at every level, reduces it from $\Theta(n \log^2 n)$ to $\Theta(n \log n)$.

Overall conclusion. The growth classes tell the whole story. Modules with logarithmic, linear or near-linear growth (S2, M2, the M4 redesign and M5) remain far below the budget of 6×10^{10} operations after the catalogue grows tenfold. The three quadratic components (M1, the current M4 design and the report generator) become a hundred times slower and exceed the nightly window by factors of 8 to 50. They must be redesigned before the expansion, and the remaining modules can safely scale to 1,000,000 products and well beyond.

Appendix :

A. Python Code for Verification of Results

```
import math
from functools import lru_cache
OPS_PER_SEC = 10**8
WINDOW_OPS = 600 * OPS_PER_SEC          # 10 minutes = 6e10 operations
# ---------- (b) M1 and M2: count by actually running the loops ----------
def m1_count(n):
    c = 0
    for i in range(0, n - 1):
        for j in range(i + 1, n):
            c += 1                      # one title+seller comparison
    return c
def m2_count(n):
    i, c = 1, 0
    while i < n:
        for j in range(i):
            c += 1                      # one Audit call
        i *= 2
    return c
for n in (10, 100, 1000):
    assert m1_count(n) == n * (n - 1) // 2
print("M1 loop count == n(n-1)/2 verified")
print("M2 n=1024:", m2_count(1024), "(formula 2^10 - 1 =", 2**10 - 1, ")")
for n in (1000, 5000):
    k = math.ceil(math.log2(n))
    assert m2_count(n) == 2**k - 1
print("M2 count == 2^ceil(log2 n) - 1 verified")
# ---------- (c), (d) recurrences solved numerically ----------
@lru_cache(None)
def S2(n): return 1 if n == 1 else S2(n // 2) + 1
@lru_cache(None)
def M4_old(n): return 1 if n == 1 else 4 * M4_old(n // 2) + n
@lru_cache(None)
def M4_new(n): return 1 if n == 1 else 2 * M4_new(n // 2) + n
@lru_cache(None)
def M5(n): return 1 if n == 1 else 2 * M5(n // 2) + n * math.log2(n)
for k in (3, 10, 16):
    n = 2**k
    assert S2(n) == k + 1
    assert M4_old(n) == 2 * n * n - n
    assert M4_new(n) == n * k + n
    assert abs(M5(n) - (n * k * (k + 1) / 2 + n)) < 1e-6 * n * k * k
print("Closed forms verified: S2, M4 old, M4 new, M5")
n = 2**20
print("Doubling ratio M4 old:", M4_old(2*n) / M4_old(n))
print("Doubling ratio M4 new:", M4_new(2*n) / M4_new(n))
# ---------- (e) S1 average case ----------
def s1_avg(n, p): return (n * (2 - p) + p) / 2
print("S1 average n=1e6, p=0.5:", s1_avg(10**6, 0.5))
# ---------- (f) Theta(n^2) check for f(n) ----------
f = lambda n: 3*n*n + 10*n*math.log2(n) + 50
```

```
assert all(3*n*n <= f(n) <= 63*n*n for n in range(1, 100000))
assert all(f(n) <= 4*n*n for n in range(64, 100000))
print("3n^2 <= f(n) <= 63n^2 (n>=1) and f(n) <= 4n^2 (n>=64) verified")
# ---------- (g) scalability table ----------
def table(n):
    L = math.log2(n)
    k = math.ceil(L)
    return {
        "M1 detector":      n * (n - 1) / 2,
        "M2 rate limiter":  2**k - 1,
        "S2 binary search": math.floor(L) + 1,
        "M4 current":       2 * n * n - n,
        "M4 redesign":      n * L + n,
        "M5 analytics":     n * L * (L + 1) / 2 + n,
        "Report f(n)":      f(n),
    }
for n in (10**5, 10**6):
    print(f"\n n = {n:,}")
    print(f"{'Module':18}{'Operations':>18}{'Seconds':>14}   Fits 10 min?")
    for name, ops in table(n).items():
        print(f"{name:18}{ops:18.3e}{ops/OPS_PER_SEC:14.4g}    {'YES' if ops <= WINDOW_OPS
else 'NO'}")
```

Output:

```
M1 loop count == n(n-1)/2 verified
M2 n=1024: 1023 (formula 2^10 - 1 = 1023 )
M2 count == 2^ceil(log2 n) - 1 verified
Closed forms verified: S2, M4 old, M4 new, M5
Doubling ratio M4 old: 4.000000953674771
Doubling ratio M4 new: 2.0952380952380953
S1 average n=1e6, p=0.5: 750000.25
3n^2 <= f(n) <= 63n^2 (n>=1) and f(n) <= 4n^2 (n>=64) verified

 n = 100,000
Module                  Operations       Seconds   Fits 10 min?
M1 detector              5.000e+09            50   YES
M2 rate limiter          1.311e+05      0.001311   YES
S2 binary search         1.700e+01       1.7e-07   YES
M4 current               2.000e+10           200   YES
M4 redesign              1.761e+06       0.01761   YES
M5 analytics             1.472e+07        0.1472   YES
Report f(n)              3.002e+10         300.2   YES

 n = 1,000,000
Module                  Operations       Seconds   Fits 10 min?
M1 detector              5.000e+11          5000   NO
M2 rate limiter          1.049e+06       0.01049   YES
S2 binary search         2.000e+01         2e-07   YES
M4 current               2.000e+12         2e+04   NO
M4 redesign              2.093e+07        0.2093   YES
M5 analytics             2.096e+08         2.096   YES
Report f(n)              3.000e+12         3e+04   NO
```

B. Optimised M1 Code Using Hashing

```python
def find_duplicates(products):
    seen, dups = set(), []
    for idx, p in enumerate(products):
        key = (p["title"], p["seller"])
        if key in seen:
            dups.append(idx)      # MarkDuplicate(idx)
        else:
            seen.add(key)
    return dups
```